

**Universidad Politécnica Internacional**

**Técnicas de Programación**  
**Documentación Técnica**

**Sistema de Gestión de Quinielas Mundialistas**  
**“Quiniela Mundial”**

**Estudiante:**  
**Glen Gerardo Villagra Rodríguez**

**Profesor:**  
**Luis Felipe Mora Umaña**

**Segundo Cuatrimestre del 2026**

|                                                              |   |
|--------------------------------------------------------------|---|
| 1. INTRODUCCIÓN .....                                        | 2 |
| 2. DECISIONES DE DISEÑO Y ARQUITECTURA.....                  | 2 |
| 2.1. Arquitectura en Capas y Patrón MVC .....                | 2 |
| 2.2. Persistencia de Datos (CSV).....                        | 2 |
| 2.3. Gestión de Estado de Sesión.....                        | 2 |
| 2.4. Control de Excepciones .....                            | 3 |
| 3. GUÍA DE EJECUCIÓN E INTERACCIÓN DEL PROGRAMA.....         | 3 |
| 3.1. Requisitos de Entorno.....                              | 3 |
| 3.2. Flujo de Operación Módulo por Módulo.....               | 3 |
| 4. ANÁLISIS DE RESULTADOS, APRENDIZAJES Y CONCLUSIONES ..... | 3 |

## 1. INTRODUCCIÓN

El presente informe expone la documentación técnica y funcional del **Sistema de Gestión de Quinielas Mundialistas**, una solución de software desarrollada sobre la plataforma .NET utilizando C# y la tecnología de interfaz gráfica Windows Forms.

El objetivo primordial del proyecto es proveer una plataforma intuitiva y estructurada donde los usuarios puedan interactuar en entornos de competencia deportiva (quinielas públicas y privadas), registrar pronósticos sobre partidos oficiales, acumular puntuación de forma automatizada y obtener insignias de rendimiento en tiempo real.

El diseño del software responde a criterios de alta cohesión, bajo acoplamiento y extensibilidad, garantizando que el núcleo de la aplicación funcione como una base sólida para futuras fases de desarrollo.

## 2. DECISIONES DE DISEÑO Y ARQUITECTURA

Con el propósito de mantener un código legible, mantenible y alineado con los principios **SOLID** y **Clean Code**, la solución se estructuró mediante los siguientes patrones y decisiones arquitectónicas:

### 2.1. Arquitectura en Capas y Patrón MVC

Se estableció una división estricta de responsabilidades en tres capas principales:

- **Vistas (Views):** Formularios interactivos (FrmLogin, FrmMenuPrincipal, FrmPartidos, FrmPronostico, FrmQuiniela, FrmEstadisticas, FrmGestionUsuarios) dedicados únicamente a la captura de eventos e interacción visual con el usuario.
- **Controladores / Servicios (Controllers / Services):** Clases donde reside la lógica del negocio (UsuarioController, PartidoController, PronosticoController, InsigniaController, NotificacionController). Encargadas de procesar reglas de puntuación, filtrado y validaciones.
- **Modelos (Models):** Clases del dominio (Usuario, Partido, Quiniela, Pronostico, Notificacion, Insignia) que representan los datos de la aplicación de manera tipada.

## 2.2. Persistencia de Datos (CSV)

En lugar de depender de un motor de base de datos relacional externo para esta primera iteración, la persistencia se manejó mediante lectura y escritura de archivos planos en formato .csv delimitados por punto y coma (;), ubicados en el directorio Data/. El procesamiento se realizó mediante el uso eficiente de las clases StreamReader y StreamWriter del espacio de nombres System.IO.

## 2.3. Gestión de Estado de Sesión

Para mantener la información del usuario autenticado a lo largo de los distintos formularios sin necesidad de acoplar la navegación pasando objetos de un Form a otro, se implementó una clase estática / utilitaria (SesionActual). Esta conserva en memoria la instancia activa durante el ciclo de vida de la ejecución.

## 2.4. Control de Excepciones

La capa de servicios valida las condiciones operativas (por ejemplo, evitar la modificación o registro de pronósticos sobre partidos que no estén en estado Pendiente). Ante un incumplimiento de regla de negocio, se lanzan excepciones que la capa de presentación captura mediante bloques try-catch, desplegando mensajes claros mediante MessageBox sin comprometer la estabilidad del sistema.

# 3. GUÍA DE EJECUCIÓN E INTERACCIÓN DEL PROGRAMA

## 3.1. Requisitos de Entorno

- Entorno de Desarrollo Integrado (IDE): Visual Studio 2022 o superior.
- SDK: .NET Desktop Development / .NET Framework compatible con Windows Forms.

### 3.2. Flujo de Operación Módulo por Módulo

1. **Autenticación (FrmLogin):** El usuario ingresa sus credenciales. El controlador valida la existencia del registro en usuarios.csv e inicializa SesionActual.
2. **Navegación Principal (FrmPrincipal):** Panel central desde donde se tiene acceso a los distintos módulos del software según el requerimiento.
3. **Registro de Pronósticos (FrmPronostico):** El usuario selecciona un encuentro en estado Pendiente y registra el marcador proyectado. El sistema guarda la asociación entre el IdUsuario y el IdPartido.
4. **Administración de Encuentros (FrmPartidos):** Permite consultar la jornada deportiva. Al actualizar un partido al estado Finalizado, el sistema dispara de forma interna el recálculo de puntuación comparando los marcadores oficiales contra los pronósticos guardados.
5. **Comunidades y Ligas (FrmQuiniela):** Permite la creación de grupos (públicos o privados) y la incorporación de participantes. Adicionalmente, incluye la funcionalidad de generación y consulta del historial de novedades (*Timeline*) en ventanas emergentes mediante StringBuilder.
6. **Módulo de Estadísticas y Reportes (FrmEstadisticas):** Muestra vistas consolidadas como el Ranking Global, el Historial de Pronósticos, métricas de partidos y la Tabla de Posiciones por grupo (ingresando la letra del grupo en el campo de texto correspondiente). También incluye la activación manual de la evaluación de insignias.

### 4. ANÁLISIS DE RESULTADOS, APRENDIZAJES Y CONCLUSIONES

- **Resultados:** Se obtuvo una aplicación funcional y reactiva capaz de aplicar las reglas de asignación de puntos de forma precisa (+5 puntos por acertar el marcador exacto y +2 puntos por acertar la tendencia del ganador o empate). La sincronización entre los controladores y la actualización de los archivos de persistencia opera correctamente en tiempo de ejecución.
- **Aprendizajes:** La aplicación del principio de Responsabilidad Única (Single Responsibility Principle) facilitó el mantenimiento de la solución. Aislar los algoritmos de cálculo dentro de los controladores permitió modificar las interfaces gráficas sin alterar el comportamiento de la lógica del negocio.
- **Conclusiones:** La arquitectura en capas elegida, sumada a la modularidad aplicada en las clases de servicio, asegura que el sistema sea completamente extensible y esté preparado de forma óptima para incorporar nuevas características o migraciones de base de datos en la siguiente iteración del proyecto.